Windows Endpoint Performance & Reliability Troubleshooting (Boot, Profile, Crash, Disk)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary 4

The business problem . . . 4

The solution . . . 4

What is included . . . 4

The outcome . . . 4

Solution Overview 5

The problem this solves . . . 5

The performance & reliability model . . . 5

Signals to measure . . . 5

The root-cause tree . . . 6

How you work through it . . . 7

Requirements 7

R1. Local administrator access on the endpoint (or Intune script/remediation delivery) . . . 7

R2. Windows Event Log access (Diagnostics-Performance, System, Application) . . . 8

R3. Performance counter access . . . 8

R4. Storage / SMART health readability . . . 8

R5. Intune Endpoint analytics (where available) . . . 9

R6. PowerShell 5.1+ and script execution . . . 9

R7. Microsoft Graph access for scope targeting (Set-ScopeTargeting only) . . . 9

Diagnostic Playbook 10

Branch A — Slow boot ("it takes forever to turn on / get to the login screen") . . . 10

Branch B — Slow sign-in / profile ("it's slow from login to a usable desktop") . . . 11

Branch C — High CPU / RAM / disk ("it's slow while I'm working") . . . 12

Branch D — Crashes / bugchecks ("it blue-screens" / "an app keeps dying") . . . 13

Branch E — Storage / SSD health (underlies A, B, C) . . . 13

After any branch — confirm and record . . . 14

Testing — Measured Pass/Fail Gates 14

T1. Prerequisite readiness gates . . . 15

T2. Boot performance gate (Branch A) . . . 15

T3. Sign-in / profile gate (Branch B) . . . 15

T4. Steady-state resource gates (Branch C) . . . 16

T5. Stability gate (Branch D) . . . 16

T6. Storage health gate (Branch E) . . . 16

T7. Whole-solution acceptance . . . 17

Troubleshooting 17

The scripts report "NOT READY" / prerequisite failures . . . 17
Boot timing is empty (no event 100) . . . 18
Sign-in duration unavailable . . . 18
Performance counters return errors or nonsense . . . 18
Storage health / SMART shows nothing . . . 18
Remediation appears to "do nothing" . . . 19
Set-ScopeTargeting cannot connect to Graph . . . 19
The fix didn't hold / the symptom returned . . . 19
Best Practices — Baselining & Repeatable Performance Triage 19
1. Baseline before you troubleshoot . . . 19
2. Measure first, always . . . 20
3. Isolate to one named cause . . . 20
4. Fix safely and reversibly . . . 20
5. Pilot before you scale . . . 20
6. Re-measure and record . . . 21
7. Escalate hardware honestly . . . 21
8. Automate the repeatable parts . . . 21
Deliverables Summary 21
Documents . . . 21
PowerShell scripts . . . 22
How to use them together . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary

The business problem

"My computer is slow." "It keeps crashing." These are the most common — and most expensive — end-user tickets, precisely because no one can quantify them. They get closed with a reboot or a costly reimage, the user loses productive hours, and the problem returns. Multiply that across a fleet and unmeasured performance and reliability issues become a steady, invisible drain on both IT time and workforce productivity.

The solution

A **measure-first root-cause method** for Windows endpoint performance and reliability in an enterprise, Intune-managed fleet. Instead of guessing, you follow a repeatable loop — **measure it, isolate it, fix it, re-measure it** — that turns a vague complaint into a specific, measured cause and a targeted fix.

The method covers the full endpoint lifecycle where slowness and instability actually live:

- **Slow boot** — measured boot timing and the exact slow driver/service behind it.
- **Slow sign-in / profile** — measured sign-in duration, profile size, and startup-app load.
- **High CPU / RAM / disk** — steady-state resource pressure with the named process responsible.
- **Crashes / bugchecks** — system Stop codes and repeat-offending applications, from the event record.
- **Storage / SSD health** — failing, worn, or full drives caught before they take a machine down.

What is included

- A **Solution Overview** defining the performance/reliability model, the signals to measure, and the root-cause tree.
- A **Requirements Guide** with a verify step for every prerequisite.
- A **Diagnostic Playbook** — the symptom tree with exact measurements and pass/fail gates per branch.
- **Five PowerShell scripts** — signal discovery, per-device measurement validation, a fleet report, safe remediation of the common causes (startup, temp, profile), and a scoped-targeting builder. Discovery/Validation/Report are read-only; Remediation and targeting default to a safe preview.
- **Testing** (measured pass/fail gates), **Troubleshooting**, and **Best Practices** for baselining and repeatable triage.

The outcome

A slow or crashing endpoint becomes a **measured cause and a targeted fix** — with a repeatable, evidence-based method and scripts the team keeps and reuses. Tickets close on facts, not guesses; hardware is replaced only when the numbers prove it; and a rising fleet trend is caught before users

complain. Self-guided and tested, at a fraction of the cost of an outside engagement.

Solution Overview

The problem this solves

"My computer is slow." "It keeps crashing." "It takes forever to log in." These are the tickets no one can quantify. They get closed with a reboot, a reimage, or a shrug — and they come back. This solution replaces guesswork with a **measure-first root-cause method**: you turn a vague complaint into a measured cause and a targeted fix, on an enterprise, Intune-managed Windows fleet.

The core discipline is simple and repeatable: **measure it** → **isolate it** → **fix it.** You never change anything until a signal tells you what is actually wrong, and you re-measure to confirm the fix worked.

The performance & reliability model

Every Windows endpoint moves through the same lifecycle each day. Slowness or instability lives in one (or more) of these stages. The model gives you a place to look for every complaint:

1. **Boot** — from power-on to the sign-in screen. The OS loads the kernel, drivers, and services.

Slow boot usually means a slow disk, a stalling driver, or too many auto-start services.

1. **Sign-in / profile** — from entering credentials to a usable desktop. The user profile loads,

Group Policy / Intune policy applies, logon scripts and startup apps run, and (for roaming/FSLogix or OneDrive) the profile container mounts. Slow sign-in usually means a large or corrupt profile, heavy startup apps, or a policy/script that blocks.

1. **Steady-state resource use** — the desktop is up and the user is working. Here you measure

CPU, memory (RAM), and disk pressure. A runaway process, a memory leak, or a disk pinned at 100% makes everything feel slow.

1. **Stability** — does the machine stay up? Crashes come in two flavors: **application crashes**

(one app dies, logged as Application Error / .NET Runtime / AppHang) and **system crashes / bugchecks** (the blue screen / Stop error — the whole OS goes down, a memory dump is written).

1. **Storage / SSD health** — underlying all of the above. A failing or worn drive, a nearly full

disk, or a drive with no free space for paging makes boot, sign-in, and steady-state all slow, and can cause crashes. This is a physical layer you must rule out early.

Signals to measure

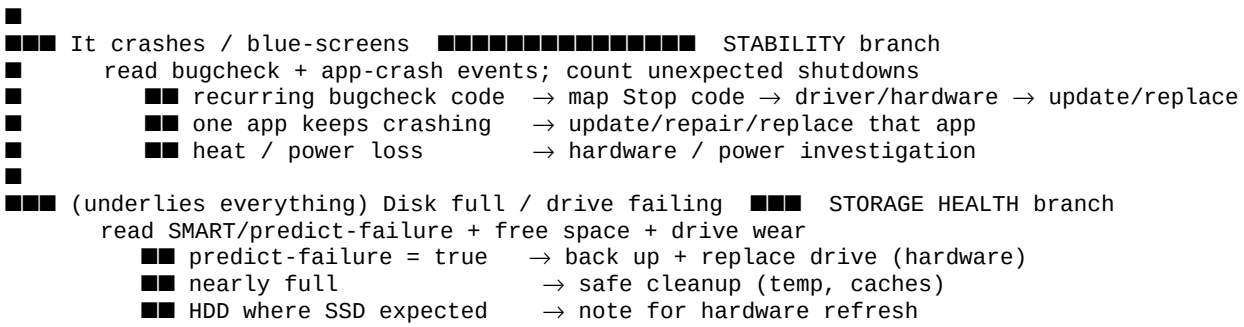
Each stage has concrete, measurable signals. This solution's scripts collect them; the playbook tells you which one to read for which symptom.

Stage	Primary signals	Where it comes from
Boot	Boot duration; time to sign-in screen; slow-service / slow-driver warnings	Event log **Diagnostics-Performance/Operational** (event IDs 100–110); Intune **Endpoint analytics** → **Startup performance** where available
Sign-in / profile	Sign-in duration; profile size; number of startup apps; slow logon events	Endpoint analytics; **User Profile Service** log; `Win32_UserProfile`; startup app inventory
Resource — CPU	% processor time; top processes by CPU	Performance counters; `Get-Process` / `Get-Counter`
Resource — RAM	Committed / available memory; top processes by working set; page-file pressure	`Get-Counter` memory counters; `Get-Process`
Resource — Disk	Disk % active time; disk queue length; free space	Performance counters; `Get-Volume`
Stability	Bugcheck (Stop) events; app-crash / app-hang events; unexpected-shutdown count; reliability index	Event log **System** (ID 41, 1001), **Application** (1000, 1002); `Win32_ReliabilityRecords`
Storage health	SMART / predict-failure status; drive wear; free-space %; media type (SSD/HDD)	`Get-PhysicalDisk` (`HealthStatus`, `MediaType`); `MSStorageDriver_FailurePredictStatus`; `Get-Volume`

The root-cause tree

Start from the symptom the user reports, then follow the branch to a measured cause. The full click-by-click measurements live in **03 — Diagnostic Playbook**; this is the map.

```
"It's slow" / "it crashes"
■
■■■ Slow to turn on / reach login  ■■■■■■■■■■■■  BOOT branch
■      measure boot duration + slow-service/driver events
■         ■■ slow disk at boot        → Storage health branch
■         ■■ a service/driver stalls  → disable/update that service or driver
■         ■■ too many auto-starts     → trim startup services
■
■■■ Slow from login to usable desktop  ■■■■■■■■  SIGN-IN / PROFILE branch
■      measure sign-in duration + profile size + startup-app count
■         ■■ large/corrupt profile    → profile remediation
■         ■■ heavy startup apps       → trim startup apps
■         ■■ blocking script/policy   → fix the logon script / policy
■
■■■ Slow while working  ■■■■■■■■■■■■■■■■■■■■■■  RESOURCE branch
■      measure CPU / RAM / disk over a window; find the top consumer
■         ■■ one process pins CPU     → update/replace/kill that app
■         ■■ memory exhausted/leak    → add RAM / fix leaking app
■         ■■ disk pinned at 100%      → Storage health branch / trim disk I/O
```



How you work through it

The whole solution follows one loop, applied per device and per fleet:

Assess → Measure → Isolate → Fix → Re-measure.

1. **Assess** — confirm you can read the endpoint's signals (see **02 — Requirements**).
2. **Measure** — run **WINPERF-Discovery.ps1** (fleet signals) and **WINPERF-Validation.ps1**

(one device, deep) to collect the numbers.

1. **Isolate** — follow the root-cause tree in **03 — Diagnostic Playbook** to the branch the numbers

point to.

1. **Fix** — apply the targeted fix. For the safe, common causes (startup bloat, temp-file disk

pressure, profile issues) use **WINPERF-Remediation.ps1** (which defaults to a preview / `-WhatIf` mode so you see what it would do before it does it).

1. **Re-measure** — re-run Validation and confirm the signal moved past the pass gate in

05 — Testing. If it did not, you isolated the wrong branch — go back to step 3.

A slow or crashing endpoint becomes a measured cause and a targeted fix — with an evidence-based method and scripts you keep and reuse.

Requirements

Before you can measure a Windows endpoint, you need to be able to *reach* its signals. This section lists every prerequisite specific to performance and reliability troubleshooting. The **category-level Global Requirements** (Entra tenant, Intune subscription, admin roles, enrollment, pilot group, supported Windows, network access) are in **global-requirements-infrastructure.md** — confirm those first, then the ones below.

Each requirement uses the same four-part format: **Description / Why / Verify / Implement if missing.**

R1. Local administrator access on the endpoint (or Intune script/remediation delivery)

Description: Either interactive local-admin rights on the device under investigation, or the ability to run PowerShell on it remotely via Intune (Platform scripts / Remediations) or a remote tool.

Why: Reading event logs, performance counters, storage/SMART data, and profile information, and applying any fix, requires elevation. Most of this solution's measurements need administrator context.

Verify: On the device, open **Windows PowerShell** as administrator and run: `([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)` — it returns `True`. For remote delivery, confirm in **Intune** → **Devices** → **Scripts and remediations** that you can create and assign a script.

Implement if missing: Add your account to the device's local **Administrators** group (or use an Entra role that grants it), or deliver the scripts through **Intune** → **Devices** → **Scripts and remediations** (which runs as SYSTEM, an elevated context).

R2. Windows Event Log access (Diagnostics-Performance, System, Application)

Description: Read access to the endpoint's event logs, specifically **Microsoft-Windows-Diagnostics-Performance/Operational**, **System**, and **Application**.

Why: Boot/sign-in timing (event IDs 100–110), bugchecks (System ID 41 and 1001), and application crashes/hangs (Application ID 1000/1002) all come from these logs. They are the primary evidence for the Boot, Sign-in, and Stability branches.

Verify: On the device run `Get-WinEvent -LogName 'Microsoft-Windows-Diagnostics-Performance/Operational' -MaxEvents 1` and `Get-WinEvent -LogName System -MaxEvents 1` — both return an event without an access error.

Implement if missing: Run elevated (see R1). If the Diagnostics-Performance log is empty or disabled, enable it: `wevtutil sl "Microsoft-Windows-Diagnostics-Performance/Operational" /e:true`, then let the device go through a normal boot cycle so events are written.

R3. Performance counter access

Description: The ability to read live Windows performance counters (Processor, Memory, PhysicalDisk, System) via `Get-Counter`.

Why: The steady-state Resource branch measures CPU %, available/committed memory, and disk % active time / queue length by sampling counters over a window. Without counter access you cannot quantify "it's slow while I work."

Verify: Run `Get-Counter '\Processor(_Total)\% Processor Time' -MaxSamples 1` — it returns a sample value.

Implement if missing: Ensure the **Performance Logs & Alerts** / **Performance Counter DLL Host** services are not disabled, and that your account is in the local **Performance Monitor Users** group (or is an administrator). If counters are corrupt, rebuild them: `lodctr /R` (elevated).

R4. Storage / SMART health readability

Description: The Storage subsystem exposes physical-disk health, media type, and failure-prediction (`Get-PhysicalDisk`, and the `MSStorageDriver_FailurePredictStatus` WMI class).

Why: The Storage-health branch must rule out a failing or worn drive and low free space — a physical cause that makes boot, sign-in, and steady-state all slow and can cause crashes. This is the layer you check early.

Verify: Run `Get-PhysicalDisk | Select FriendlyName, MediaType, HealthStatus` — it lists the drive(s) with a **HealthStatus** (Healthy / Warning / Unhealthy). Then `Get-CimInstance -Namespace root\wmi -ClassName MSStorageDriver_FailurePredictStatus` returns a `PredictFailure` value on most physical hardware.

Implement if missing: Update the storage controller / disk drivers so the health APIs are populated. On some virtual machines SMART is not surfaced — record "SMART unavailable (virtual)" rather than a false pass. Ensure at least ~15% free space so cleanup and paging have room.

R5. Intune Endpoint analytics (where available)

Description: Endpoint analytics enabled in Intune, providing **Startup performance** (boot + sign-in timing) and **Application reliability** across the fleet.

Why: Endpoint analytics gives you a *fleet-wide, historical* view of boot and sign-in times and crash-prone apps — the baseline you compare a single slow device against. It turns "is this device slow?" into "is this device slow *relative to the fleet*?"

Verify: In **Intune** → **Reports** → **Endpoint analytics**, the **Startup performance** and **Application reliability** tabs load with data. (Data appears a few days after onboarding.)

Implement if missing: In **Intune** → **Reports** → **Endpoint analytics** → **Settings**, enable Endpoint analytics and choose the device scope (all Intune-managed devices, or a group). Wait for the initial data collection. **Endpoint analytics requires the appropriate Intune/Windows Enterprise licensing** — if it is unavailable in your tenant, this solution's per-device scripts still deliver the same measurements locally; you simply lose the pre-built fleet baseline and read it from the **WINPERF-Report.ps1** roll-up instead.

R6. PowerShell 5.1+ and script execution

Description: Windows PowerShell 5.1 (built into supported Windows) or PowerShell 7, with a policy that allows the solution's read-only scripts to run.

Why: All five scripts are PowerShell. Discovery, Validation, and Report are read-only; Remediation and Set-ScopeTargeting default to a preview (`-WhatIf`) mode.

Verify: Run `$PSVersionTable.PSVersion` (expect 5.1 or higher) and `Get-ExecutionPolicy`.

Implement if missing: Run scripts with `powershell -ExecutionPolicy Bypass -File .\script.ps1` (per-process, changes nothing permanently), or set a signed-script policy via Intune. When delivering through Intune Remediations, execution policy is handled by the platform.

R7. Microsoft Graph access for scope targeting (Set-ScopeTargeting only)

Description: The ability to sign in to Microsoft Graph with a scope that can read and manage Entra group membership (`Group.ReadWrite.All` or equivalent), used only by **WINPERF-Set-ScopeTargeting.ps1**.

Why: The scope-targeting script builds and populates the pilot / rollout Entra security group used to target measurement or remediation at a controlled set of devices. This keeps you piloting before fleet-wide action, per the deployment model.

Verify: Run `Connect-MgGraph -Scopes 'Group.ReadWrite.All'` and confirm it connects with the consented scope (the **Microsoft.Graph** PowerShell module must be installed: `Get-Module Microsoft.Graph -ListAvailable`).

Implement if missing: Install the module — `Install-Module Microsoft.Graph -Scope CurrentUser` — and have an Intune/Entra administrator consent to the `Group.ReadWrite.All` scope. The script defaults to `-WhatIf`, so you can validate the plan before any group is changed.

Diagnostic Playbook

This is the heart of the solution: a **symptom tree** that turns a complaint into a measured cause and a targeted fix. Start from the user's own words, pick the matching branch, run the measurement, read it against the gate, and act. Every branch ends with a **re-measure** step so you prove the fix.

The loop, every time: Measure → Isolate → Fix → Re-measure.

Run the two collectors first so you have numbers in front of you:

- **WINPERF-Discovery.ps1** — quick, read-only fleet/device sweep of the signals.
- **WINPERF-Validation.ps1** — deep, per-device measurement (this is what each branch below reads).

If you have Intune **Endpoint analytics** (requirement R5), also open **Reports** → **Endpoint analytics** → **Startup performance / Application reliability** to see how this device compares to the fleet baseline.

Branch A — Slow boot ("it takes forever to turn on / get to the login screen")

What to measure

1. Boot duration from the Diagnostics-Performance log:

`Get-WinEvent -LogName 'Microsoft-Windows-Diagnostics-Performance/Operational' -FilterXPath "*[System[(EventID=100)]]" -MaxEvents 5` — event **100** carries `BootTime` (total) and `MainPathBootTime` in milliseconds.

1. Slow contributors — same log, event IDs **101** (slow app), **102** (slow driver), **103** (slow

service), **106** (slow background optimization), **109** (slow device).

1. Boot-critical disk — is the OS on an SSD or HDD? `Get-PhysicalDisk | Select MediaType, HealthStatus`.

How to read it (pass/fail gate)

- **PASS:** `BootTime` ≤ 60,000 ms (60 s) on an SSD device, and no repeated 102/103 slow warnings.
- **FAIL:** `BootTime` > 60 s, or the same driver (102) / service (103) appears slow across multiple

boots.

Isolate → fix

• **Slow disk at boot** (HDD, or SSD `HealthStatus` not Healthy) → go to **Branch E (Storage health)**;

the disk is the cause.

• **A specific driver stalls (102)** → note the driver name; update it (vendor / Windows Update /

Intune driver policy). If a recently added driver, roll it back.

• **A specific service stalls (103)** → identify the service (`Get-Service the-service-name`); if it is a

non-essential third-party auto-start, set it to **Automatic (Delayed Start)** or disable it. The safe, common case (trimming non-essential auto-start startup items) is handled by **WINPERF-Remediation.ps1 -Area Startup** (preview first with `-WhatIf`).

• **Too many auto-starts (many 101/103)** → trim startup services/apps (Remediation, Startup area).

Re-measure: reboot twice (the first reboot re-baselines), re-read event 100. Confirm `BootTime` now meets the PASS gate.

Branch B — Slow sign-in / profile ("it's slow from login to a usable desktop")

What to measure

1. Sign-in duration — from **Endpoint analytics** → **Startup performance** (it separates *core boot* from

sign-in / *time to first responsive desktop*). If analytics is unavailable, use the User Profile Service log: `Get-WinEvent -LogName 'Microsoft-Windows-User Profile Service/Operational' -MaxEvents 40` and look for long gaps between profile-load start and finish.

1. Profile size — `Get-CimInstance Win32_UserProfile | Select LocalPath, @{n='SizeGB';e={...}}` (the

Validation script computes this); a very large local profile slows load.

1. Startup-app count — inventory `HKCU/HKLM ...\Run`, the Startup folders, and scheduled logon tasks

(the Validation script lists these).

1. Blocking logon scripts / policies — long-running GPO/Intune logon scripts.

How to read it (pass/fail gate)

• **PASS:** sign-in ≤ 30 s to a responsive desktop; profile a reasonable size for the environment;

a manageable startup-app count (roughly ≤ 10 auto-start user apps as a working default — set your own from the fleet baseline).

• **FAIL:** sign-in > 30 s, an oversized/growing profile, or a large pile of startup apps.

Isolate → fix

• **Large or corrupt profile** → if roaming/FSLogix/OneDrive Known Folder Move is in play, check the

container/sync health first. For a corrupt local profile, back up user data and recreate the profile (the Remediation script's **Profile** area handles the safe, reversible parts — clearing stale temp/cache within the profile — and flags a full profile rebuild for manual action).

- **Heavy startup apps** → disable non-essential ones (**Remediation -Area Startup**, preview first).
- **Blocking logon script/policy** → find the slow script from the User Profile Service / GroupPolicy

logs and make it asynchronous or remove it.

Re-measure: sign the user out and back in; re-check sign-in duration against the PASS gate.

Branch C — High CPU / RAM / disk ("it's slow while I'm working")

This branch measures **steady-state resource pressure** and names the top consumer. It samples over a window so a momentary spike is not mistaken for a chronic problem.

What to measure

1. Sample counters over ~60 seconds (the Validation script does this):
   - CPU: `\Processor(_Total)\% Processor Time`
   - Memory: `\Memory\% Committed Bytes In Use`, `\Memory\Available MBytes`
   - Disk: `\PhysicalDisk(_Total)\% Disk Time`, `\PhysicalDisk(_Total)\Current Disk Queue Length`
1. Top consumers: `Get-Process | Sort CPU -Descending | Select -First 5` and `Get-Process | Sort WorkingSet -Descending | Select -First 5`.

How to read it (pass/fail gate)

- **CPU PASS:** average `% Processor Time` < 70% over the window with no single non-system process

holding the CPU. **FAIL:** sustained ≥ 85%, or one app pinning a core continuously.

- **RAM PASS:** `% Committed Bytes In Use` < 80% and Available MBytes comfortably above zero.

FAIL: committed ≥ 90% / available near zero (constant paging).

- **Disk PASS:** average `% Disk Time` < 80% and queue length < 2 sustained. **FAIL:** disk pinned at

~100% with a growing queue.

Isolate → fix

- **One process pins CPU** → identify it; update or reconfigure the app, or (if hung) end it. If it is

a Windows/AV process peaking during a scan, confirm it settles after the scan.

- **Memory exhausted / leak** → find the process whose working set climbs without release; update or

replace it. If all apps are legitimate and RAM is simply insufficient, record a **hardware RAM upgrade** recommendation with the measured evidence.

- **Disk pinned at 100%** → find the process driving I/O. If the drive itself is slow/failing, go to

Branch E (Storage health). If a background task (search indexing, backup, AV scan) is the cause, reschedule it.

Re-measure: after the fix, re-sample the counters over another 60 s window; confirm the failing counter now meets its PASS gate.

Branch D — Crashes / bugchecks ("it blue-screens" / "an app keeps dying")

Separate the two crash types — they have different causes and fixes.

What to measure

1. System crashes / bugchecks:

`Get-WinEvent -LogName System -FilterXPath "*[System[(EventID=1001)]]"` — the **BugCheck** event (1001) records the **Stop code** and parameters. Event **41** (Kernel-Power) counts *unexpected* shutdowns (power loss / hard hang).

1. Application crashes/hangs:

`Get-WinEvent -LogName Application -FilterXPath "*[System[(EventID=1000 or EventID=1002)]]"` — **1000** = application error (faulting module), **1002** = application hang.

1. Reliability history: `Get-CimInstance Win32_ReliabilityRecords` for a timeline and the reliability

index trend (also visible in **Endpoint analytics** → **Application reliability**).

How to read it (pass/fail gate)

- **PASS:** no bugcheck (1001) in the last 14 days and no repeated same-app crash (1000/1002).
- **FAIL:** any bugcheck in the window, or the same faulting module/app crashing repeatedly.

Isolate → fix

- **Recurring bugcheck code** → map the **Stop code** to its cause (most bugchecks point to a driver

or failing hardware). Note the faulting driver in the event; update or roll it back. If the code points to memory/storage, run hardware diagnostics and go to **Branch E**.

- **Same app keeps crashing** → note the **faulting module** in event 1000; update or repair the app

(or its runtime, e.g. .NET / Visual C++ redistributable). Replace if unfixable.

- **Unexpected shutdowns (event 41 with no bugcheck)** → suspect power (battery/adapter/PSU) or thermal

shutdown; investigate hardware/power and check for overheating.

Re-measure: after the fix, monitor for the crash-free window (the PASS gate) before closing. Bugchecks are proven fixed only by absence over time.

Branch E — Storage / SSD health (underlies A, B, C)

Rule this out early — a failing, worn, or full drive causes slow boot, slow sign-in, high disk %, and crashes all at once.

What to measure

1. Physical health: `Get-PhysicalDisk | Select FriendlyName, MediaType, HealthStatus, OperationalStatus`.
2. Failure prediction (SMART):

`Get-CimInstance -Namespace root\wmi -ClassName MSStorageDriver_FailurePredictStatus` — a `PredictFailure = True` is an early-warning that the drive is dying.

1. Free space: `Get-Volume | Select DriveLetter, FileSystemLabel, SizeRemaining, Size` — compute

free %.

1. Media type: SSD vs HDD (`MediaType`) — is a device that *should* have an SSD still on an HDD?

How to read it (pass/fail gate)

- **PASS:** `HealthStatus = Healthy`, `PredictFailure = False`, free space ≥ 15% of the volume.
- **FAIL:** HealthStatus Warning/Unhealthy, PredictFailure True, or free space < 15%.

Isolate → fix

- **PredictFailure = True / HealthStatus not Healthy** → this is **hardware**. Back up the user's data immediately, then schedule a **drive replacement**. Do not "fix" a dying drive in software.
- **Nearly full (< 15% free)** → safe cleanup: clear Windows and user temp, empty recycle bin, clear browser/app caches, run the Storage Sense / cleanup tasks. **WINPERF-Remediation.ps1 -Area Temp** does the safe cleanup (preview with `-WhatIf` first, so you see the reclaimable space before deleting).
- **HDD where an SSD is expected** → record a **hardware refresh** recommendation with the measured boot/sign-in impact as justification.

Re-measure: re-run **WINPERF-Validation.ps1** and confirm free space / health now pass, and that the boot/disk symptoms that pointed here have improved.

After any branch — confirm and record

1. Re-run the branch's measurement and confirm it meets the **PASS** gate in **05 — Testing**.
2. If it still fails, you picked the wrong branch — return to the root-cause tree in

01 — Solution Overview and follow the next-most-likely branch.

1. Record the device, the symptom, the measured cause, the fix, and the before/after numbers. This is your baseline for next time and feeds the fleet report (**WINPERF-Report.ps1**).

Testing — Measured Pass/Fail Gates

Troubleshooting is only "done" when a measurement proves it. This section is the single source of **pass/fail gates**: run the check, read the value, compare to the gate. Use it twice — once to confirm the *cause* (a failing gate), and again after the fix to confirm the *cure* (a passing gate).

The gates below are sensible enterprise defaults. Where you have an Intune **Endpoint analytics** baseline (requirement R5), tune each threshold to your fleet's own distribution and record the value you chose.

T1. Prerequisite readiness gates

Run before any measurement. All must PASS or the numbers cannot be trusted.

Check	Command / location	PASS
Elevation	`IsInRole(Administrator)` (R1)	`True`
Diagnostics-Performance log readable	`Get-WinEvent -LogName '...Diagnostics-Performance/Operational' -MaxEvents 1`	returns an event
System / Application logs readable	`Get-WinEvent -LogName System -MaxEvents 1`	returns an event
Counters readable	`Get-Counter '\Processor(_Total)\% Processor Time' -MaxSamples 1`	returns a sample
Storage health readable	`Get-PhysicalDisk`	lists disk(s) with HealthStatus
PowerShell version	`$PSVersionTable.PSVersion`	≥ 5.1

WINPERF-Discovery.ps1 performs T1 automatically and reports READY / NOT READY.

T2. Boot performance gate (Branch A)

Metric	Source	PASS	FAIL
`BootTime`	Diag-Perf event 100	≤ 60,000 ms (SSD)	> 60,000 ms
Repeated slow driver (102)	Diag-Perf	none recurring	same driver across boots
Repeated slow service (103)	Diag-Perf	none recurring	same service across boots

Re-test after **two** reboots (the first re-baselines).

T3. Sign-in / profile gate (Branch B)

Metric	Source	PASS	FAIL
Sign-in to responsive desktop	Endpoint analytics / User Profile Service log	≤ 30 s	> 30 s
Startup app count	Run keys + Startup folders + logon tasks	≤ 10 (default)	well above baseline
Profile size	`Win32_UserProfile`	reasonable for fleet	oversized / growing

Re-test with a fresh sign-out / sign-in.

T4. Steady-state resource gates (Branch C)

Sampled over a ~60-second window.

Resource	Counter	PASS	FAIL
CPU	`\Processor(_Total)\% Processor Time` (avg)	< 70%	≥ 85% sustained
Memory	`\Memory\% Committed Bytes In Use`	< 80%	≥ 90%
Memory	`\Memory\Available MBytes`	comfortably > 0	near 0 (constant paging)
Disk	`\PhysicalDisk(_Total)\% Disk Time` (avg)	< 80%	~100% sustained
Disk	`\PhysicalDisk(_Total)\Current Disk Queue Length`	< 2 sustained	≥ 2 sustained

A single named process may not exceed the gate on its own for a PASS.

T5. Stability gate (Branch D)

Metric	Source	PASS	FAIL
Bugchecks (Stop)	System event 1001	none in last 14 days	any in window
Unexpected shutdowns	System event 41	none unexplained	recurring
Repeated app crash/hang	Application event 1000 / 1002	no repeated faulting module	same module repeating

A bugcheck fix passes only by **absence over the 14-day window**, not immediately.

T6. Storage health gate (Branch E)

Metric	Source	PASS	FAIL
Health status	`Get-PhysicalDisk` HealthStatus	Healthy	Warning / Unhealthy
Failure prediction	`MSStorageDriver_FailurePredictStatus`	False	True
Free space	`Get-Volume` (free ÷ size)	≥ 15%	< 15%

A FAIL on failure prediction or health status is a **hardware** replacement, not a software fix.

T7. Whole-solution acceptance

The engagement passes when:

1. Every device in scope has been run through **WINPERF-Discovery** + **WINPERF-Validation** and has a

recorded T1 READY result.

1. Each reported symptom maps to a **named branch and a measured cause** (no "unknown / rebooted it").

2. Every applied fix has a **before/after** measurement showing the failing gate now passes (or, where

the cause is hardware, a documented replacement/upgrade recommendation with the evidence).

1. **WINPERF-Report.ps1** produces a fleet roll-up with no unexplained FAIL rows.

Troubleshooting

This section is for when the **diagnostic tooling itself** does not behave — the measurements you rely on in the playbook are missing, empty, or throwing errors. Fix these first, or the numbers you read will be wrong. (For fixing the *device's* performance, use **03 — Diagnostic Playbook**; this is about fixing your ability to *measure* it.)

The scripts report "NOT READY" / prerequisite failures

Symptom: WINPERF-Discovery.ps1 prints NOT READY, or a script errors on access.

- **Not elevated.** Re-launch PowerShell **as administrator** (R1). Verify with the `IsInRole`

check in **05 — Testing** → **T1**. When running via Intune Scripts/Remediations you run as SYSTEM (already elevated).

- **Execution policy blocks the script.** Launch with

`powershell -ExecutionPolicy Bypass -File .\WINPERF-Discovery.ps1` (per-process, permanent nothing).

- **Wrong PowerShell version.** `$PSVersionTable.PSVersion` must be ≥ 5.1.

Boot timing is empty (no event 100)

Symptom: the Diagnostics-Performance query returns nothing.

- The **Diagnostics-Performance/Operational** log is disabled. Enable it:

`wevtutil sl "Microsoft-Windows-Diagnostics-Performance/Operational" /e:true`, then perform a normal power-off/power-on cycle (not just a reboot) so boot events are written; re-query afterwards.

- The device has not had a full boot recently — event 100 is written at boot. Reboot and re-check.
- On some VMs, boot performance eventing is limited; note "boot timing unavailable (virtual)" rather than recording a false pass, and lean on the Sign-in and Resource branches.

Sign-in duration unavailable

Symptom: no Endpoint analytics sign-in data, or the User Profile Service log is empty.

- **Endpoint analytics not onboarded / not licensed** (R5). Use the local User Profile Service log instead: `Get-WinEvent -LogName 'Microsoft-Windows-User Profile Service/Operational'`. If that log is also empty, enable it via `wevtutil` and re-test after a sign-out/sign-in.
- Analytics data lags onboarding by a few days — if you just enabled it, wait for collection.

Performance counters return errors or nonsense

Symptom: `Get-Counter` throws, or values are obviously wrong (e.g., negative, or always 0).

- **Corrupt counters.** Rebuild them elevated: `lodctr /R` (and, for WMI-exposed counters, `winmgmt /resyncperf`), then retry.
- **Localized counter names.** On non-English Windows, counter paths differ. Use the language-neutral index form or run the Validation script, which resolves counters by ID.
- **Service disabled.** Ensure **Performance Counter DLL Host** / **Performance Logs & Alerts** are not disabled.

Storage health / SMART shows nothing

Symptom: `Get-PhysicalDisk` HealthStatus is blank, or `MSStorageDriver_FailurePredictStatus` throws.

- **Virtual disk / VM** — SMART is not passed through. Record "SMART unavailable (virtual)"; rely on free-space and disk-% signals instead. Do not report a false PASS.
- **Storage driver too old** — update the storage controller/disk driver so the health APIs populate.

- **Namespace/class access** — run elevated; the `root\wmi` class requires administrator context.

Remediation appears to "do nothing"

Symptom: WINPERF-Remediation.ps1 runs but nothing changes.

- **This is by design.** Remediation **defaults to** `-WhatIf` — it *previews* changes without making them. Read the preview output; when satisfied, re-run in apply mode (drop `-WhatIf` / confirm).
- **Nothing met the safety criteria** — the script only touches non-essential startup items, safe temp paths, and profile caches. If there is nothing safe to clean, it correctly reports "no action".

Set-ScopeTargeting cannot connect to Graph

Symptom: WINPERF-Set-ScopeTargeting.ps1 fails at `Connect-MgGraph`.

- **Module missing** — `Install-Module Microsoft.Graph -Scope CurrentUser` (R7).
- **Scope not consented** — an admin must consent to `Group.ReadWrite.All`. Re-run `Connect-MgGraph -Scopes 'Group.ReadWrite.All'`.
- **Preview mode masking intent** — the script defaults to `-WhatIf`; it will *describe* the group changes without making them. Confirm the plan, then re-run in apply mode.

The fix didn't hold / the symptom returned

- **You isolated the wrong branch.** Re-run **WINPERF-Validation.ps1** and follow the next-most-likely branch in the root-cause tree (**01 — Solution Overview**).
- **The real cause is hardware** (failing drive, insufficient RAM, thermal/power). Software fixes will not stick — deliver the measured hardware recommendation instead (see **07 — Best Practices §7**).
- **A fleet-wide cause** (a bad driver or app in the image) reappears per device — fix it at the image / policy level, not one machine at a time.

Best Practices — Baselining & Repeatable Performance Triage

Troubleshooting one machine is firefighting. The value of this solution compounds when you make it **repeatable** and **baselined**, so the next slow-PC ticket takes minutes and every fix is evidence- based. These are the habits that turn the playbook into an operating standard.

1. Baseline before you troubleshoot

- **Establish a fleet baseline first.** Run **WINPERF-Discovery.ps1** (or read Endpoint analytics)

across a healthy sample and record the *normal* ranges for boot time, sign-in time, startup-app count, and steady-state CPU/RAM/disk. "Slow" only means something relative to normal.

- **Tune the gates in 05 — Testing to your baseline.** The defaults are starting points. Replace them with your fleet's real numbers and write the chosen values down.
- **Re-baseline after major changes** — a Windows feature update, a new image, a new security agent — because these shift the normal ranges.

2. Measure first, always

- **Never change anything before a measurement points at it.** A reboot that "fixes" nothing wastes a troubleshooting opportunity and the evidence.
- **Sample over a window, not a moment.** A one-second spike is not a chronic problem; the 60-second counter window in the Resource branch exists to avoid false alarms.
- **Rule out storage early.** A failing or full disk masquerades as many different symptoms — check Branch E health/free-space before spending time in Boot, Sign-in, or Resource.

3. Isolate to one named cause

- **Follow the tree to a single branch.** Resist the urge to apply three fixes at once — you will not know which one worked, and you will not learn.
- **Name the culprit** (this driver, this service, this process, this drive), not a category. "A service is slow" is not actionable; "the *XYZ Sync* service adds 22 s to boot" is.

4. Fix safely and reversibly

- **Always preview remediation.** Run **WINPERF-Remediation.ps1** with its default `-WhatIf` first and read exactly what it would change (which startup items, how much temp space, which profile caches) before committing with `-Confirm`/explicit apply.
- **Prefer the least-destructive fix.** Delayed-start a service before disabling it; clear a cache before rebuilding a profile; reschedule a background task before killing it.
- **Back up before any drive or profile action.** Storage-health FAILs and full profile rebuilds touch user data — protect it first.

5. Pilot before you scale

- **Use scoped targeting.** Build a pilot Entra group with **WINPERF-Set-ScopeTargeting.ps1** (default `-WhatIf`) and validate any fleet-wide remediation on the pilot first — Assess → Configure → Pilot → Validate → Expand.

• **Expand only after the pilot passes its gates**, then widen the group in stages.

6. Re-measure and record

• **Prove every fix with a before/after number** against the pass gate — this is the difference between "I did something" and "the problem is measurably gone."

• **Keep a per-device record** of symptom → measured cause → fix → before/after. Over time this becomes your fastest triage tool: recurring causes jump out.

• **Feed patterns back into the baseline.** If the same driver or app crashes across many devices, that is a fleet action (update the image, block the app), not a per-device fix.

7. Escalate hardware honestly

• A drive predicting failure, insufficient RAM proven by the memory gate, or an HDD where an SSD is expected are **hardware** outcomes. Deliver them as a **measured recommendation** (with the numbers), not as a software workaround — a worn drive cannot be fixed in software.

8. Automate the repeatable parts

• Deliver **Discovery** and **Validation** through **Intune** → **Scripts and remediations** on a schedule so you have current signals *before* the ticket arrives.

• Run **WINPERF-Report.ps1** periodically to watch the fleet trend — a rising boot-time or crash rate is a problem forming, caught before users complain.

Deliverables Summary

Everything included in this solution, and how the pieces fit the **measure** → **isolate** → **fix** → **re-measure** method.

Documents

Document	What it gives you
01 — Solution Overview	The performance/reliability model (boot → sign-in/profile → steady-state resource use → stability → storage health), the signals to measure, and the root-cause tree.
02 — Requirements	Every prerequisite (endpoint/admin access, event & counter access, storage/SMART readability, Endpoint analytics where available, Graph for targeting) with Description / Why / Verify / Implement-if-missing.

Document	What it gives you
Global Requirements — Infrastructure	The category-level prerequisites (Entra tenant, Intune, admin roles, enrollment, pilot group, supported Windows, network).
03 — Diagnostic Playbook	The symptom tree with exact measurements and pass/fail gates per branch: slow boot, slow sign-in/profile, high CPU/RAM/disk, crashes/bugchecks, storage/SSD health.
05 — Testing	The consolidated measured pass/fail gates — used to confirm both the cause and the cure.
06 — Troubleshooting	What to do when the tooling itself does not behave (no events, corrupt counters, SMART unavailable, remote-run failures).
07 — Best Practices	Baselining and repeatable performance triage as an operating standard.

PowerShell scripts

All scripts share a standard header (PURPOSE / HOW TO RUN / EXPECTED RESULTS). Discovery, Validation, and Report are **read-only**. Remediation and Set-ScopeTargeting default to a **preview (`-WhatIf`)** mode so nothing changes until you decide.

Script	Role in the method	Read-only?
WINPERF-Discovery.ps1	Quick sweep of performance/reliability signals; confirms prerequisites (T1) and flags where to look.	Yes
WINPERF-Validation.ps1	Deep per-device measurement — boot, sign-in/profile, CPU/RAM/disk over a window, crashes, storage health — evaluated against the pass/fail gates.	Yes
WINPERF-Report.ps1	Fleet-wide performance/reliability roll-up from collected results; highlights FAIL rows and trends.	Yes
WINPERF-Remediation.ps1	Safe, reversible fixes for the common causes — startup bloat, temp/disk pressure, profile cache — **defaults to** `-WhatIf`.	Preview by default

Script	Role in the method	Read-only?
WINPERF-Set-ScopeTargeting.ps1	Builds/populates the pilot or rollout Entra group via Microsoft Graph so measurement and remediation are scoped — **defaults to** `-WhatIf`.	Preview by default

How to use them together

1. **WINPERF-Discovery.ps1** across the fleet → who is slow / unstable, and are prerequisites ready.
2. **WINPERF-Set-ScopeTargeting.ps1** → put the affected devices in a scoped pilot group.
3. **WINPERF-Validation.ps1** on a device → the measured cause, mapped to a playbook branch.
4. **WINPERF-Remediation.ps1 -WhatIf** → preview the safe fix, then apply.
5. **WINPERF-Validation.ps1** again → re-measure and confirm the gate now passes.
6. **WINPERF-Report.ps1** → record the outcome and watch the fleet trend.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

WINPERF-Discovery.ps1

Purpose: Collects a fast snapshot of the key performance and reliability signals on a Windows endpoint (boot timing, resource pressure, recent crashes, storage health) and confirms the prerequisites needed to measure deeper. Read-only: it changes nothing. Use it first to see who is slow or unstable and where to look next. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WINPERF-Discovery.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\WINPERF-Discovery.ps1

WINPERF-Remediation.ps1

Purpose: Applies the safe, common fixes surfaced by the Diagnostic Playbook: trimming non-essential startup items, clearing safe temp/cache paths to reclaim disk space, and clearing stale profile caches. It DEFAULTS TO PREVIEW (-WhatIf) — it shows exactly what it would change and how much space it would reclaim, and changes nothing until you re-run in apply mode. HOW TO RUN Preview (default, safe): powershell -ExecutionPolicy Bypass -File .\WINPERF-Remediation.ps1 -Area Temp Apply for real: powershell -ExecutionPolicy Bypass -File .\WINPERF-Remediation.ps1 -Area Temp -Apply

How to run: Preview (default, safe): powershell -ExecutionPolicy Bypass -File .\WINPERF-Remediation.ps1 -Area Temp Apply for real: powershell -ExecutionPolicy Bypass -File .\WINPERF-Remediation.ps1 -Area Temp -Apply

WINPERF-Report.ps1

Purpose: Aggregates the JSON result files produced by WINPERF-Discovery.ps1 and WINPERF-Validation.ps1 across many devices into a single fleet view: how many devices pass each gate, which devices are slow or unstable, and the FAIL rows that need attention. Read-only: it only reads the result files you point it at and writes a report. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WINPERF-Report.ps1 -InputFolder .\results

How to run: powershell -ExecutionPolicy Bypass -File .\WINPERF-Report.ps1 -InputFolder .\results

WINPERF-Set-ScopeTargeting.ps1

Purpose: Creates or updates the Microsoft Entra security group used to scope performance measurement and remediation to a controlled pilot (then a wider rollout), keeping you piloting before any fleet-wide action. It DEFAULTS TO PREVIEW (-WhatIf) — it describes the group and membership changes it would make and changes nothing until you re-run in apply mode. HOW TO RUN Preview (default, safe): powershell -ExecutionPolicy Bypass -File .\WINPERF-Set-ScopeTargeting.ps1 -GroupName "WINPERF-Pilot" -MemberDeviceNames pc-01,pc-02 Apply for real: powershell -ExecutionPolicy Bypass -File .\WINPERF-Set-ScopeTargeting.ps1 -GroupName "WINPERF-Pilot" -MemberDeviceNames pc-01,pc-02 -Apply

How to run: Preview (default, safe): powershell -ExecutionPolicy Bypass -File .\WINPERF-Set-ScopeTargeting.ps1 -GroupName "WINPERF-Pilot" -MemberDeviceNames pc-01,pc-02 Apply for real: powershell -ExecutionPolicy Bypass -File .\WINPERF-Set-ScopeTargeting.ps1 -GroupName "WINPERF-Pilot" -MemberDeviceNames pc-01,pc-02 -Apply

WINPERF-Validation.ps1

Purpose: Performs the full per-device measurement behind the Diagnostic Playbook: boot timing and slow contributors, sign-in/profile signals, steady-state CPU/RAM/disk sampled over a window, crash/bugcheck history, and storage/SSD health. Scores each against the pass/fail gates and maps failures to a playbook branch. Read-only: it measures, it does not change anything. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WINPERF-Validation.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\WINPERF-Validation.ps1